

CS 300 Pseudocode Document

Course Structure

STRUCT Course

 STRING courseNumber

 STRING courseTitle

 LIST<String> prerequisites

END STRUCT

// Vector - Milestone 1

FUNCTION loadVector(fileName, Vector<Course> courses)

 Open fileName for reading

 IF file cannot be opened THEN

 Print "Error: Unable to open file"

 Return false

 END IF

 Create an empty list called allCourseNumbers

 Create an empty list called courseRecords

 WHILE file has another line

 Read one line

 Split line by commas into fields

 IF number of fields is less than 2 THEN

 Print "Error: Invalid file format"

 Close file

```
    Return false

END IF

courseNumber = trim(first field)

courseTitle = trim(second field)

IF courseNumber is empty OR courseTitle is empty THEN

    Print "Error: Missing course number or title"

    Close file

    Return false

END IF

Add courseNumber to allCourseNumbers

Add fields to courseRecords

END WHILE

Close file

FOR each record in courseRecords

    FOR each prerequisite field after the second field

        IF prerequisite is not found in allCourseNumbers THEN

            Print "Error: Prerequisite course does not exist"

            Return false

        END IF

    END FOR

END FOR

FOR each record in courseRecords
```

Create Course course

course.courseNumber = first field

course.courseTitle = second field

FOR each remaining field

 Add prerequisite to course.prerequisites

END FOR

 Add course to courses vector

END FOR

Return true

END FUNCTION

Search and Print One Course

FUNCTION searchCourse(Vector<Course> courses, STRING courseNumber)

 FOR each course in courses

 IF course.courseNumber equals courseNumber THEN

 Print course.courseNumber

 Print course.courseTitle

 IF course.prerequisites is empty THEN

 Print "Prerequisites: None"

 ELSE

 Print "Prerequisites:"

 FOR each prerequisiteNumber in course.prerequisites

 FOR each prerequisiteCourse in courses

```
    IF prerequisiteCourse.courseNumber equals prerequisiteNumber THEN

        Print prerequisiteCourse.courseNumber

        Print prerequisiteCourse.courseTitle

        BREAK

    END IF

END FOR

END FOR

END IF

Return

END IF

END FOR

Print "Course not found"

END FUNCTION

Print Sorted Course List

FUNCTION printVectorCourseList(Vector<Course> courses)

    Copy courses into sortedCourses

    Sort sortedCourses by courseNumber from lowest to highest

    FOR each course in sortedCourses

        Print course.courseNumber

        Print course.courseTitle

    END FOR

END FUNCTION

// Hash Table - Milestone 2
```

FUNCTION loadHashTable(fileName, HashTable<Course> courses)

Open fileName for reading

IF file cannot be opened THEN

Print "Error: Unable to open file"

Return false

END IF

Create allCourseNumbers

Create courseRecords

WHILE file has another line

Read one line

Split line by commas

IF number of fields is less than 2 THEN

Print "Error: Invalid file format"

Close file

Return false

END IF

courseNumber = trim(first field)

courseTitle = trim(second field)

IF courseNumber is empty OR courseTitle is empty THEN

Print "Error: Missing course information"

Close file

Return false

```
END IF

Add courseNumber to allCourseNumbers

Add fields to courseRecords

END WHILE

Close file

FOR each record in courseRecords

    FOR each prerequisite field

        IF prerequisite is not found in allCourseNumbers THEN

            Print "Error: Prerequisite course does not exist"

            Return false

        END IF

    END FOR

END FOR

FOR each record in courseRecords

    Create Course course

    Store course number, title, and prerequisites

    key = hash(course.courseNumber)

    Insert course into hash table using key

    IF collision occurs THEN

        Store course in the same bucket using separate chaining

    END IF

END FOR
```

Return true

END FUNCTION

//Search and Print One Course

FUNCTION searchCourse(HashTable<Course> courses, STRING courseNumber)

key = hash(courseNumber)

course = search bucket using key

IF course is not found THEN

Print "Course not found"

Return

END IF

Print course.courseNumber

Print course.courseTitle

IF course.prerequisites is empty THEN

Print "Prerequisites: None"

ELSE

Print "Prerequisites:"

FOR each prerequisiteNumber in course.prerequisites

prerequisiteKey = hash(prerequisiteNumber)

prerequisiteCourse = search bucket using prerequisiteKey

IF prerequisiteCourse is found THEN

Print prerequisiteCourse.courseNumber

Print prerequisiteCourse.courseTitle

```
        END IF

    END FOR

END IF

END FUNCTION

//Print Sorted Course List

FUNCTION printHashTableCourseList(HashTable<Course> courses)

    Create empty Vector<Course> sortedCourses

    FOR each bucket in the hash table

        FOR each course in the bucket

            Add course to sortedCourses

        END FOR

    END FOR

    Sort sortedCourses by courseNumber from lowest to highest

    FOR each course in sortedCourses

        Print course.courseNumber

        Print course.courseTitle

    END FOR

END FUNCTION

// Binary Search Tree - Milestone 3

FUNCTION loadTree(fileName, Tree<Course> courses)

    Open fileName for reading

    IF file cannot be opened THEN

        Print "Error: Unable to open file"
```



```
    Return false

END IF

Create allCourseNumbers

Create courseRecords

WHILE file has another line

    Read one line

    Split line by commas

    IF number of fields is less than 2 THEN

        Print "Error: Invalid file format"

        Close file

        Return false

    END IF

    courseNumber = trim(first field)

    courseTitle = trim(second field)


    IF courseNumber is empty OR courseTitle is empty THEN

        Print "Error: Missing course information"

        Close file

        Return false

    END IF

    Add courseNumber to allCourseNumbers

    Add fields to courseRecords

END WHILE
```

Close file

FOR each record in courseRecords

 FOR each prerequisite field

 IF prerequisite is not found in allCourseNumbers THEN

 Print "Error: Prerequisite course does not exist"

 Return false

 END IF

 END FOR

END FOR

FOR each record in courseRecords

 Create Course course

 Store course number, title, and prerequisites

 Insert course into binary search tree

END FOR

 Return true

END FUNCTION

// Insert Course into Tree

FUNCTION insertCourse(node, course)

 IF node is NULL THEN

 Create new node containing course

 Return new node

 END IF

```
IF course.courseNumber is less than node.course.courseNumber THEN

    node.left = insertCourse(node.left, course)

ELSE

    node.right = insertCourse(node.right, course)

END IF

Return node

END FUNCTION

FUNCTION searchCourse(Tree<Course> courses, STRING courseNumber)

    current = root

    WHILE current is not NULL

        IF current.course.courseNumber equals courseNumber THEN

            Print current.course.courseNumber

            Print current.course.courseTitle

            IF current.course.prerequisites is empty THEN

                Print "Prerequisites: None"

            ELSE

                Print "Prerequisites:"

                FOR each prerequisiteNumber in current.course.prerequisites

                    prerequisiteCourse = search tree for prerequisiteNumber

                    IF prerequisiteCourse is found THEN

                        Print prerequisiteCourse.courseNumber

                        Print prerequisiteCourse.courseTitle

                    
```

```
        END IF

    END FOR

END IF

Return

ELSE IF courseNumber is less than current.course.courseNumber THEN

    current = current.left

ELSE

    current = current.right

END IF

END WHILE

Print "Course not found"

END FUNCTION

// Print Sorted Course List

FUNCTION printTreeCourseList(node)

    IF node is NULL THEN

        Return

    END IF

    printTreeCourseList(node.left)

    Print node.course.courseNumber

    Print node.course.courseTitle

    printTreeCourseList(node.right)

END FUNCTION
```

// Menu Pseudocode

START PROGRAM

Create selected data structure

Set dataLoaded = false

REPEAT

 Print "1. Load Data"

 Print "2. Print Course List"

 Print "3. Print Course Information"

 Print "9. Exit"

Read userChoice

IF userChoice equals 1 THEN

 Ask user for file name

 Load file into selected data structure

 IF loading succeeds THEN

 dataLoaded = true

 Print "Courses loaded successfully"

 END IF

ELSE IF userChoice equals 2 THEN

 IF dataLoaded equals false THEN

 Print "Please load the course data first"

 ELSE

 Print courses in alphanumeric order

```
END IF

ELSE IF userChoice equals 3 THEN

    IF dataLoaded equals false THEN

        Print "Please load the course data first"

    ELSE

        Ask user for course number

        Search for course

        Print course title and prerequisites

    END IF

ELSE IF userChoice equals 9 THEN

    Print "Goodbye"

ELSE

    Print "Invalid menu option"

END IF

UNTIL userChoice equals 9

END PROGRAM
```

Runtime Analysis

The following tables evaluate the worst-case running time for reading the input file, validating prerequisite course numbers, creating Course objects, and inserting the objects into each data structure. The menu and course-list printing functions are not included in this analysis.

The Vector Runtime Analysis

Code	Line Cost	# Times Executes	Total Cost
Read and parse each course line	1	n	n
Add each course number and record	1	n	n
Validate prerequisites against course numbers	1	$n \times n$	n^2
Create each Course object	1	n	n
Append each Course to vector	1	n	n
Total Cost			$n^2 + 4n$
Runtime			$O(n^2)$

Hash Table Runtime Analysis

Code	Line Cost	# Times Executes	Total Cost
Read and parse each course line	1	n	n
Add each course number and record	1	n	n
Validate prerequisites against course numbers	1	$n \times n$	n^2
Create each Course object	1	n	n

Hash and insert each Course	$O(n)$, worst case	n	n^2
Total Cost			$2n^2 + 3n$
Runtime			$O(n^2)$

Binary Search Tree Runtime Analysis

Code	Line Cost	# Times Executes	Total Cost

Read and parse each course line	1	n	n
Add each course number and record	1	n	n
Validate prerequisites against course numbers	1	$n \times n$	n^2
Create each Course object	1	n	n
Insert each Course into an unbalanced tree	$O(n)$, worst case	n	n^2
Total Cost			$2n^2 + 3n$
Runtime			$O(n^2)$

Memory Analysis

Data Structure	Memory Usage
Vector	$O(n)$
Hash Table	$O(n)$
Binary Search Tree	$O(n)$

All three structures store one Course object for each of the n courses. The hash table also stores buckets, and the binary search tree stores left and right node pointers.

Advantages and Disadvantages

Vector

Advantages: A vector is simple to implement, easy to traverse, and stores elements in contiguous memory. Adding courses to the end is efficient.

Disadvantages: Searching for an individual course requires a linear scan in the worst case. The courses must also be sorted before the program can print them in alphanumeric order.

Hash Table

Advantages: A hash table provides fast average-case insertion and lookup. It is a strong choice when the main requirement is quickly retrieving one course by course number.

Disadvantages: A hash table does not naturally preserve alphanumeric order. To print a sorted list, the courses must be copied to another structure and sorted. Collisions also require a handling method such as separate chaining and may reduce performance.

Binary Search Tree

Advantages: A binary search tree naturally supports alphanumeric output through in-order traversal. Searching and insertion are efficient when the tree remains reasonably balanced.

Disadvantages: The tree can become unbalanced, causing search and insertion to degrade to linear time. Each node also requires additional memory for child pointers.

Recommendation

Based on the runtime analysis and the project requirements, I recommend using the Binary Search Tree. It naturally stores courses in alphanumeric order through an in-order traversal, making it efficient for printing the complete course list. It also supports efficient searching for individual courses and their prerequisites when the tree is reasonably balanced. Although a hash table offers fast average-case searches, it does not maintain sorted order without additional processing. Therefore, the Binary Search Tree provides the best overall solution for ABCU's advising system.